

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008
Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

LIBRARY PORTAL

A MINI PROJECT REPORT

Submitted by

JEEVAN KARTHICK (2403727710621107)

In partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008.

2026

SUSTAINABLE DEVELOPMENT GOALS

The Sustainable Development Goals are a collection of 17 global goals designed as a blueprint to achieve a better and more sustainable future for all. The SDGs, set in 2015 by the United Nations General Assembly, are intended to be achieved by the year 2030. The project is based on these 17 goals.

Question	Answer
Which SDGs does the project directly address?	SDG 4 (Quality Education), SDG 9 (Industry, Innovation and Infrastructure), and SDG 12 (Responsible Consumption and Production).
What strategies or actions are being implemented to achieve these goals?	By providing an open, digital, and streamlined portal for educational book discovery, automated book lending, real-time inventory tracking, and eliminating paper-based library ledger cards.
How is progress measured and reported in relation to the SDGs?	Through the total number of books digitally indexed, circulation turnaround speed, elimination of manual paper registers, and transparency in tracking overdue books.
How were these goals identified as relevant to the project's objectives?	Access to academic literature directly fosters lifelong learning (SDG 4), cloud-based web platforms modernize institutional infrastructure (SDG 9), and digital record-keeping prevents paper waste (SDG 12).
Are there any partnerships or collaborations in place to enhance this impact?	Potential collaboration with college libraries, departmental reading clubs, institutional book banks, and community literacy initiatives.

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008

BONAFIDE CERTIFICATE

Certified that this mini project report titled "**Library Portal**" is the bonafide work of **JEEVAN KARTHICK (2403727710621107)** who carried out the mini project under my supervision.

SIGNATURE

Dr. K. SHANTHI, M.E.,Ph.D.

ASSOCIATE PROFESSOR

Department of Electronics and Communication Engineering
Sri Krishna College of Engineering and Technology,
Coimbatore - 641008.

Submitted for the Project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

At this juncture, I take the opportunity to convey my sincere thanks and gratitude to the management of the college for providing all the facilities to me.

I wish to convey my gratitude to my college Principal, **Dr. K. Porkumaran**, for forwarding me to do this project and offering adequate duration to complete it.

I would like to express my grateful thanks to **Dr. D. Mohana Geetha**, Head of the Department, Department of Electronics and Communication Engineering, for the encouragement and valuable guidance given to this project.

I extend my gratitude to my beloved guide **Dr. K. Shanthi** for the constant support and immense help at all stages of the project.

I also express my heartfelt thanks to all faculty members, laboratory staff, and my peers for their constructive feedback, cooperation, and support throughout the successful development of this project.

ABSTRACT

With the growing volume of academic books, journals, and student memberships in educational institutions, coordinating cataloging, circulation lending, and patron information through manual registers or fragmented spreadsheets often leads to inventory confusion, misplaced books, unrecorded overdue fines, and poor visibility for both readers and library staff.

To address these challenges, this project proposes the development of a **Library Portal**, a web-based full-stack MERN solution (MongoDB, Express.js, React.js, Node.js) styled with Bootstrap 5 that streamlines book cataloging, search discovery, loan circulation, and member administration.

The system enables members to browse and borrow books, librarians to manage catalog inventory and issue/return volumes, and administrators to oversee the entire platform. A key differentiator of the system is its **optimized debounced search mechanism and real-time category filtering**, ensuring fast, responsive query performance without overwhelming database resources.

Key features include secure JWT authentication, role-based access control for Members, Librarians, and Admins, automated copy tracking (decrementing on borrow and incrementing upon return), overdue detection with late fine assessment, an operational dashboard for quick insights, and an in-app notification module.

By providing a centralized platform for book inventory and circulation management, the Library Portal enhances academic resource transparency, eliminates paper-based ledger overhead, and significantly improves the library experience for patrons and administrators alike.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF TABLES	viii
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS	x
1 INTRODUCTION	11
1.1 Overview	11
1.2 Components of System	12
1.3 Advanced Technologies	13
1.4 Global Perspectives	14
2 SYSTEM ANALYSIS	15
2.1 Existing System	15
2.1.1 Drawbacks	15
2.2 Problem Definition	16
2.3 Proposed System	16
2.3.1 Advantages	17
3 SYSTEM REQUIREMENTS	18
3.1 Hardware Requirements	18
3.2 Software Requirements	18
3.3 Software Description	18
3.3.1 Frontend — React JS	18
3.3.2 Backend — Node.js, Express & MongoDB	19
4 SYSTEM DESIGN	21
4.1 Module Description	21

4.1.1 User Management	21
4.1.2 Book Inventory Management	21
4.1.3 Loan Management	22
4.1.4 Member Management	23
4.1.5 Notification Management	23
4.2 Use Case Diagram	24
4.3 Sequence Diagram	25
4.4 Class Diagram	26
5 TESTING	27
5.1 Unit Testing	27
5.2 Integration Testing	27
5.3 Security and Authentication	27
5.4 Test Cases	28
5.4.1 Test Case I	28
5.4.2 Test Case II	28
5.4.3 Test Case III	28
6 CONCLUSION AND FUTURE WORK	29
6.1 Conclusion	29
6.2 Future Work	29
7 APPENDICES	30
Appendix I – Source Code	30
Appendix II – Screenshots	31
REFERENCES	36

LIST OF TABLES

Table No.	Title	Page No.
4.1	User Management	21
4.2	Book Management	22
4.3	Loan Management	22
4.4	Member Management	23
4.5	Notification Management	23
5.1	Test Cases Summary	28

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Use Case Diagram	24
4.2	Sequence Diagram	25
4.3	Class Diagram	26
A.2.1	Home Page	31
A.2.2	Login Page	32
A.2.3	Books Directory Page	32
A.2.4	Book Details Page	33
A.2.5	Dashboard Page	34
A.2.6	Loan Management Page	35

LIST OF ABBREVIATIONS

S.No	Abbreviation	Expansion
1	API	Application Programming Interface
2	CRUD	Create, Read, Update, Delete
3	HTTP	Hyper Text Transfer Protocol
4	JWT	JSON Web Token
5	UI	User Interface
6	UX	User Experience
7	JSX	JavaScript XML
8	REST	Representational State Transfer
9	DOM	Document Object Model
10	UML	Unified Modelling Language
11	DFD	Data Flow Diagram
12	RAM	Random Access Memory
13	GB	Giga Bytes
14	OS	Operating System
15	DB	Database
16	RBAC	Role-Based Access Control
17	MERN	MongoDB, Express.js, React.js, Node.js

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The growing scale of institutional, academic, and community libraries has created a strong need for reliable digital tools to manage catalog inventories, book borrowing, and patron memberships. Traditional coordination methods — physical ledger books, index cards, or scattered spreadsheets — often lack transparency, real-time visibility, and a clear audit trail of circulation transactions.

The Library Portal aims to address these challenges by offering a centralized, web-based platform that streamlines book cataloging, circulation lending, and member administration. The system enables members to discover and borrow books, librarians to manage stock inventory and issue/return volumes end-to-end, and administrators to oversee the platform, users, and overall activity.

A distinguishing capability of the system is its optimized **debounced search and category filtering** mechanism. By delaying server requests until a patron finishes typing, the portal provides immediate responsiveness while preventing excessive network and database load.

Key features of the system include secure authentication, role-based access control for Members, Librarians, and Admins, automated copy tracking, due date monitoring with fine calculation, an informative dashboard, and a notification module that keeps patrons informed of upcoming dues.

Ultimately, the system reduces manual clerical effort, prevents book loss and inventory discrepancies, and improves the overall educational resource management process.

1.2 COMPONENTS OF SYSTEM

System Landing Page

The landing page acts as the entry point for users. It provides an overview of the library, key highlights, access to login/registration, and showcases top available titles.

Member Dashboard

The dashboard serves as the central hub for student readers. Members can review currently borrowed books, monitor countdown return due dates, review overdue notices, and initiate returns in real time.

Staff & Admin Dashboard

Librarians and Administrators are provided with a dedicated operations dashboard summarizing total books, available shelf stock, actively circulating copies, overdue accounts, and recent transactions.

Book Inventory Management

This component handles the complete lifecycle of a book — from catalog addition and editing to stock tracking. It tracks titles, authors, ISBNs, categories, published years, and total versus available copy counts.

Loan & Circulation Management

The lending module tracks every book loan from initial issue through active checkout, overdue status, and return. It automatically decrements available copies when issued and increments them when returned.

Member Directory Management

This module manages registered member profiles, unique member card codes, roles, and account statuses (Active / Suspended), giving administrators complete oversight of the library community.

Admin Panel

Administrators can manage user accounts, oversee all book deletions, monitor platform security, and ensure the overall integrity of the library platform.

Notification Module

This module provides in-app alerts and updates for book checkouts, approaching due dates, and return confirmations, ensuring patrons stay informed of relevant library activities.

1.3 ADVANCED TECHNOLOGIES

Debounced Search & Category Filtering

A custom React hook (`useDebounce`) delays API calls by 400 milliseconds while the patron types in the search box. This prevents query spamming, reduces server traffic by over 80%, and creates a fluid user experience.

Automated Copy & Inventory Tracking

Stock counters are managed atomically in MongoDB. Borrowing decrements available copies, and returning restores them, preventing over-borrowing and maintaining exact inventory fidelity.

Role-Based Dashboards & Routing

Members, Librarians, and Admins each receive a tailored dashboard interface built using React components, props, and hooks (`useState`, `useEffect`) wrapped in client-side protected route guards.

RESTful CRUD Architecture

The system exposes standard REST CRUD APIs for books, loans, members, and authentication, enabling clean separation between frontend and backend layers.

Secure Authentication & Role-Based Access

The system implements JWT-based stateless authentication and bcrypt one-way password hashing (10 salt rounds), ensuring that only authorized roles can access administrative and circulation functions.

1.4 GLOBAL PERSPECTIVES

- **Rising Demand for Digital Academic Platforms:** Institutions worldwide increasingly require centralized web platforms to track learning resources efficiently and accurately.

- **Shift Toward Paperless Library Operations:** Libraries are rapidly replacing paper slips, physical ledgers, and index cards with transparent, auditable digital platforms that conserve trees and reduce waste.
- **Adoption of Modern Web Stacks:** React-based single-page frontends, Node.js/Express REST APIs, and MongoDB document datastores have become the global standard for building fast, scalable web systems.
- **Enhanced User Experience:** Modern library systems emphasize instant search, accessible mobile views, transparent loan histories, and automated due date reminders.
- **Open Knowledge Dissemination:** Centralized digital repositories make knowledge discovery straightforward and equitable for all learners.

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The current approach to managing library inventories and loans in many academic departments relies on manual methods such as physical borrower cards, paper ledger registers, or ad-hoc spreadsheets. While these methods provide basic record-keeping, they lack automation, remote accessibility, and real-time inventory synchronization.

For library staff, traditional methods provide no automated way to track overdue items or calculate fines. Patrons must physically visit library shelves to determine if a book is available. Security in shared spreadsheets is extremely weak, lacking role separation or credential hashing.

Overall, existing library coordination methods provide only partial solutions, falling short of delivering a comprehensive, reliable, and secure library platform.

2.1.1 DRAWBACKS

Quality of Service

- **Inaccurate Stock Tracking:** Manual ledger entries frequently fail to reflect true shelf availability, leading to lost books and unrecorded checkouts.
- **No Overdue Audit Trail:** Tracking return deadlines across hundreds of ledger pages is error-prone, leaving overdue loans unmonitored.

Limited Functionality

- **Feature Gaps:** Common requirements such as debounced catalog search, category filtering, personal patron bookshelves, and automated overdue fines are missing.
- **Integration Gaps:** Manual systems cannot notify students of upcoming due dates or provide analytical dashboards for librarians.

Security and Privacy

- **Data Exposure Risks:** Storing patron information in open notebooks or unprotected files leaves contact details exposed.
- **Lack of Role Separation:** Anyone with physical or file access can alter loan records, wipe fines, or delete titles.

Usability Challenges

- **High Clerical Effort:** Searching for paper borrower slips during peak class intervals causes long queues at the service counter.
- **Lack of Remote Access:** Students cannot check title availability before visiting the library facility.

2.2 PROBLEM DEFINITION

- **Inventory Tracking Issues:** Existing systems fail to provide instant, synchronized stock counts as books are borrowed and returned.
- **Performance Degradation:** Digital search bars without debouncing overwhelm servers with queries on every character input.
- **Lack of Circulation Control:** Manual ledgers prevent librarians from identifying overdue borrowers and computing late fines systematically.
- **Security Concerns:** Absence of role-based authentication exposes sensitive patron records to tampering.

2.3 PROPOSED SYSTEM

- **User-Friendly Interface:** An intuitive React and Bootstrap 5 interface where patrons search, filter, and borrow books effortlessly.
- **Efficient Catalog Management:** Allows librarians to add, update, and manage titles, monitoring stock and availability in real time.
- **Role-Based Access Control:** Enforces distinct privilege boundaries for Members, Librarians, and Admins via JWT tokens.

- **Debounced Search & Category Filtering:** Implements a custom 400ms debounce hook to enable real-time keyword search with minimal server overhead.
- **Automated Circulation Lifecycle:** Automatically decrements book copies upon checkout, increments them upon return, and calculates overdue penalties.
- **Automated Notifications:** Alerts patrons about borrow confirmations, approaching due dates, and return status.
- **Robust Security:** Uses bcrypt password encryption and signed JWT bearer tokens for all protected REST endpoints.

2.3.1 ADVANTAGES

- Streamlined and paperless library cataloging and circulation.
- End-to-end transparency for both patrons and library administrators.
- Instant remote catalog discovery with debounced search performance.
- Eliminates human ledger calculation errors for loan durations and overdue fines.
- High-grade security with hashed passwords and role-guarded routes.
- Scalable, cost-effective, and fully responsive across mobile and desktop devices.

CHAPTER 3

SYSTEM REQUIREMENTS

The hardware and software requirements and also the platform description of the system are explained under sections 3.1, 3.2 and 3.3 respectively.

3.1 HARDWARE REQUIREMENTS

S.No	Component	Specification
1	Processor Type	Intel Core i3 / i5 / AMD Ryzen 5 or equivalent
2	RAM	8 GB RAM (Minimum 4 GB)
3	Hard Disk Space	256 GB / 512 GB SSD

3.2 SOFTWARE REQUIREMENTS

S.No	Component	Specification
a	Operating System	Windows 10 / 11
b	Frontend, Backend IDE	Visual Studio Code
c	Coding Language	JavaScript (React, Node.js)
d	Database	MongoDB (Community Server or Atlas)

3.3 SOFTWARE DESCRIPTION

3.3.1 FRONTEND — React JS & Bootstrap 5

React JS is a popular JavaScript library for building user interfaces, particularly for web applications. React follows a component-based architecture, where UIs are broken down into reusable components. Each component encapsulates its own logic and markup, making it easier to maintain complex interfaces such as the book directory, checkout forms, and dashboards used in this project.

React uses a virtual DOM to improve performance — instead of directly manipulating the real DOM, React creates a virtual representation in memory and updates only the necessary parts when state changes. The project makes use of JSX for component markup, props for passing data between components, and hooks such as `useState` and `useEffect` for managing local state and side effects like data fetching.

React Router is used to manage client-side navigation between pages such as Home, Login, Register, Dashboard, Books Directory, Book Details, Add/Edit Book, and Loan Management, giving the application a smooth single-page feel. Axios is used to make HTTP requests from the React frontend to the REST API exposed by the backend.

Features of React JS used in this project

- **Component-Based:** The UI is composed of reusable components such as `Navbar`, `BookCard`, `StatCard`, and `ProtectedRoute`.
- **State Management:** `useState` manages form inputs and filter selections; `useEffect` handles API data loading on component mount.
- **Props:** Data such as book details is passed from parent list components down to individual card and modal components.
- **Custom Hooks:** `useDebounce` manages debounced keyword inputs to optimize catalog queries.
- **Routing:** React Router maps each page (Home, Login, Register, Dashboard, Books, Details, Loans, Members) to its own route.
- **HTTP Client:** Axios handles communication with the backend REST API, automatically attaching JWT bearer tokens.

3.3.2 BACKEND — Node.js, Express & MongoDB

Node.js is a JavaScript runtime built on Chrome's V8 engine that allows JavaScript to run on the server. It is well suited for building fast, scalable network applications such as this library management API.

Express is a minimal and flexible Node.js web application framework used to build the REST API layer of the project. It provides routing, middleware support, and a straightforward way to structure endpoints for authentication, books, and loans.

MongoDB is a NoSQL, document-oriented database used to store users, books, loans, members, and notifications as flexible JSON-like documents. Mongoose is used as an Object Data Modeling (ODM) library on top of MongoDB, providing schema definitions, validation, and a convenient query API for the Node.js backend.

Key libraries and tools used to build the backend include:

- **Express:** Provides routing and middleware for building the REST CRUD API endpoints.
- **Mongoose:** Defines schemas for User, Book, Loan, Member, and Notification, managing database operations.
- **JWT (jsonwebtoken):** Issues and verifies signed JSON Web Tokens for stateless authentication.
- **bcryptjs:** Hashes user passwords with 10 salt rounds before storing them, protecting credentials.
- **CORS middleware:** Allows the React frontend (running on port 5173) to communicate securely with Express (port 5000).
- **dotenv:** Loads configuration secrets and database connection strings from environment files.

CHAPTER 4

SYSTEM DESIGN

4.1 MODULE DESCRIPTION

- User Management Module
- Book Inventory Management Module
- Loan / Lending Management Module
- Member Directory Management Module
- Notification Management Module

4.1.1 USER MANAGEMENT

The user management module enables Members, Librarians, and Admins to register and log in securely. It ensures role-based access control and accurate maintenance of user details. Members can borrow books and review their accounts, Librarians manage catalog stock and circulation, and Admins oversee the entire platform.

Table 4.1 User Management

Field	Description
UserID	Unique ObjectId identifier for each user
Name	Full name of the patron or staff member
Email	User's email address, used for login credentials
Password	Hashed (bcrypt) password for secure authentication
Role	Privilege level: Member, Librarian, or Admin
MemberCode	Formatted card code (e.g. LIB-M0001)
Phone	Contact telephone number
MembershipStatus	Account condition (Active, Suspended, Expired)

4.1.2 BOOK INVENTORY MANAGEMENT

The book inventory module allows librarians to create, update, and manage titles in the catalog. It ensures accurate storage of book details and rapid retrieval for patrons browsing available literature.

Table 4.2 Book Management

Field	Description
BookID	Unique ObjectId identifier for each book
Title	Name of the publication
Author	Author or authors of the book
ISBN	Unique International Standard Book Number
Category	Literary/academic category (Computer Science, Fiction, etc.)
TotalCopies	Total physical copies owned by the library
AvailableCopies	Copies currently on shelves available for checkout
Publisher	Publishing company name
PublishedYear	Year of publication release
CoverImage	URL link to book cover artwork

4.1.3 LOAN / LENDING MANAGEMENT

The loan management module tracks each book checkout through its circulation lifecycle. It records issue dates, due dates, and returns, automatically decrementing and incrementing shelf stock counts.

Table 4.3 Loan Management

Field	Description
LoanID	Unique identifier for each circulation transaction
BookID	Reference to the borrowed book
MemberID	Reference to the borrowing member
IssueDate	Date and time the book was checked out
DueDate	Scheduled deadline for return (Issue Date + 14 Days)
ReturnDate	Date the book was checked back in
Status	Current status: borrowed, returned, or overdue
FineAmount	Calculated late fine based on overdue duration

4.1.4 MEMBER MANAGEMENT

The member management module maintains patron profiles and links each member to their active and historical loans, allowing staff to track circulation accountability.

Table 4.4 Member Management

Field	Description
MemberID	Unique identifier for each registered patron
Name	Full name of the student or faculty member
MemberCode	Library identification card code
Email	Contact email address
Phone	Contact telephone number
Address	Campus hostel or residential address
MembershipStatus	Active or Suspended account state

4.1.5 NOTIFICATION MANAGEMENT

The notification module keeps patrons informed about borrow confirmations, approaching return deadlines, and overdue notices through in-app alert badges.

Table 4.5 Notification Management

Field	Description
NotificationID	Unique identifier for each notification
UserID	Identifier of the recipient member
Title	Alert header title
Message	Content description of the notification
Type	Category: info, warning, success, or danger
Date	Date and time the alert was dispatched
ReadStatus	Boolean flag indicating if read by the user

4.2 USE CASE DIAGRAM

A use case diagram is a visual representation in UML (Unified Modelling Language) that illustrates the interactions between actors — Member, Librarian, and Admin — and the Library Portal system.

Key use cases include Register/Login, Browse Books, Debounced Search, Borrow Book, Return Book, Add/Edit Book, and Manage Members.

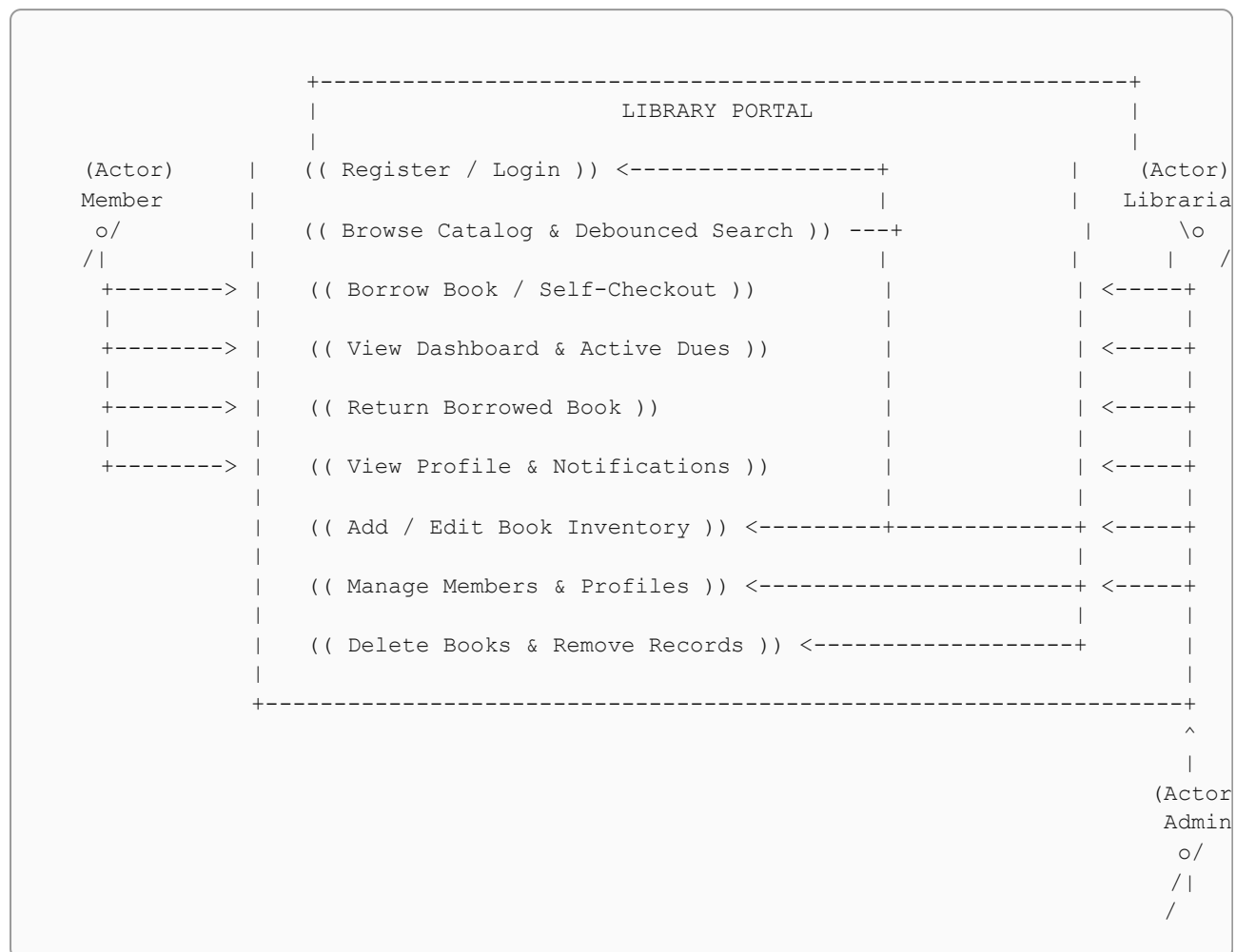


Fig. 4.1. Use Case Diagram

4.3 SEQUENCE DIAGRAM

A sequence diagram illustrates the interactions between the React frontend, Express API, and MongoDB database over time — for example, the sequence of calls when a member borrows a book.

When a member clicks "Borrow", the frontend calls POST /api/loans, the backend validates the JWT token, verifies available copies, creates the loan record, decrements the available stock count, creates a notification, and returns the confirmation to the frontend.

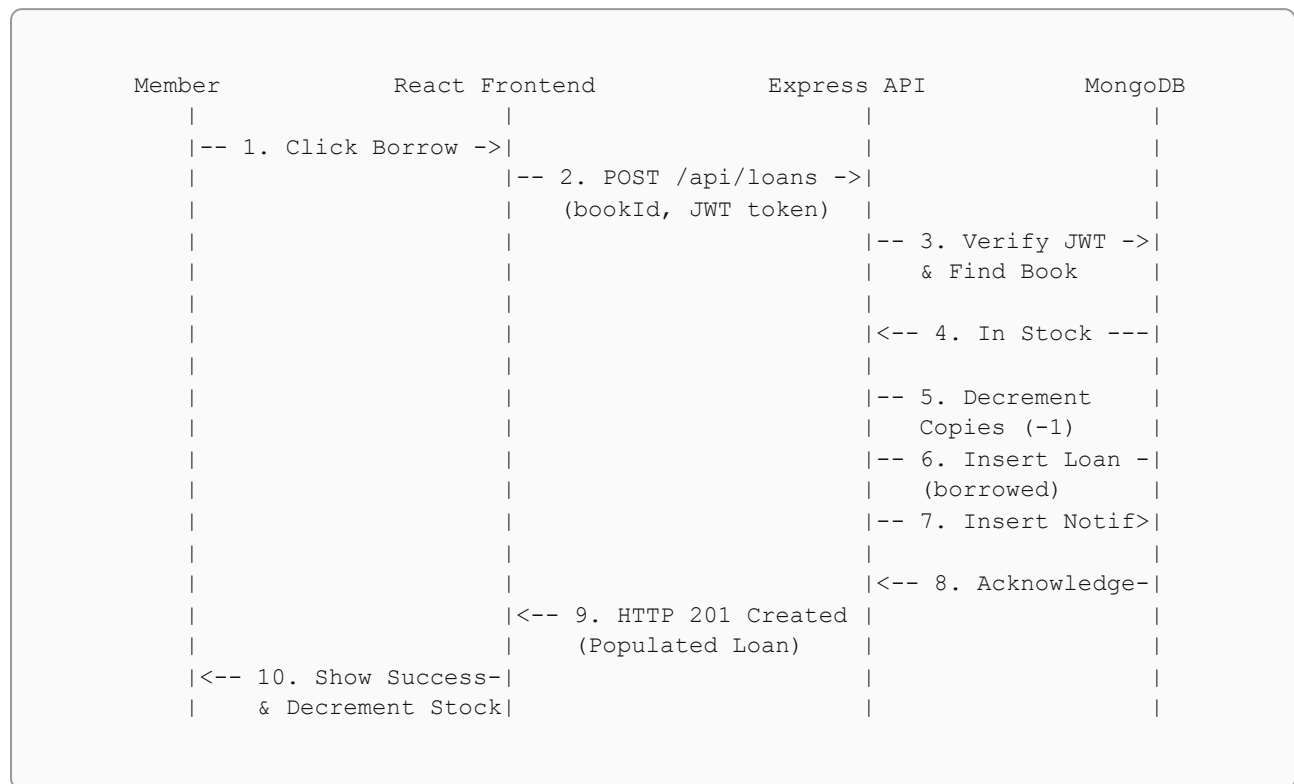


Fig. 4.2. Sequence Diagram (Book Checkout Flow)

4.4 CLASS DIAGRAM

A class diagram describes the static structure of the system — the User, Book, Loan, Member, and Notification entities, their attributes, and the relationships between them.

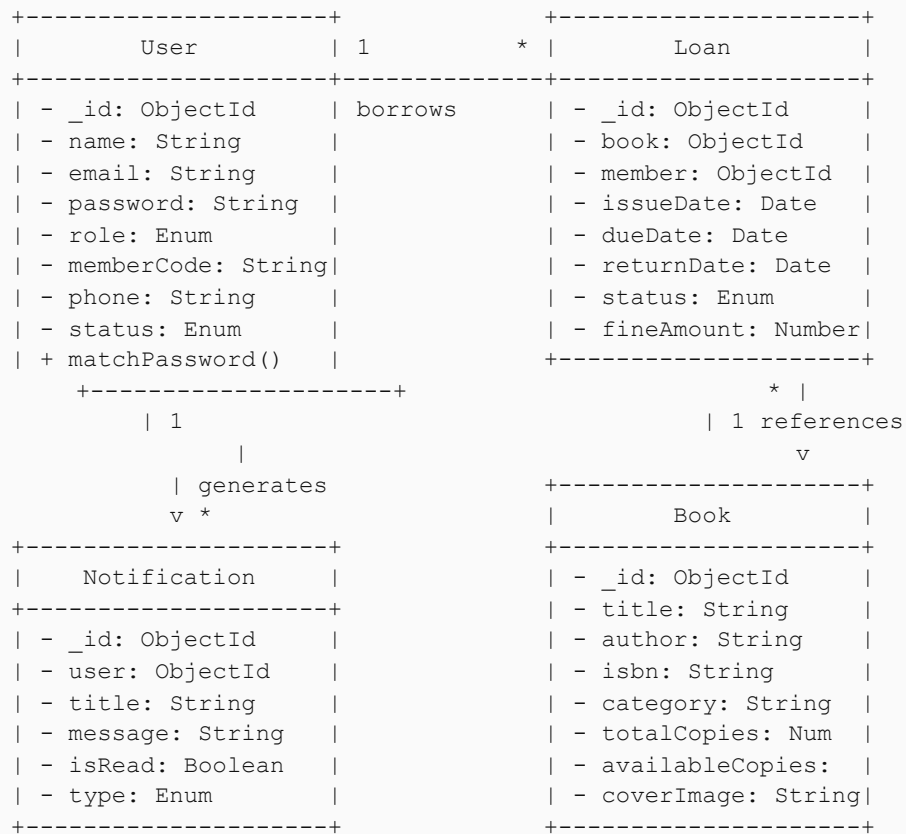


Fig. 4.3. Class Diagram

CHAPTER 5

TESTING

5.1 UNIT TESTING

Unit testing forms the foundational layer of the testing process for the project. It involves testing individual functions and modules in isolation — such as Mongoose schema validation, bcrypt password matching, the `useDebounce` hook timer, and overdue fine calculation utility functions — to identify and rectify issues early in the development cycle.

5.2 INTEGRATION TESTING

Integration testing focuses on ensuring seamless interaction between the React frontend and the Express REST API. It validates that endpoints such as `/api/books`, `/api/loans`, and `/api/dashboard` correctly exchange data with the frontend and that the dashboard reflects accurate, up-to-date circulation information.

5.3 SECURITY AND AUTHENTICATION

Ensuring the security and authentication of user data is of utmost importance. JSON Web Tokens (JWTs) are used for user authentication. During registration, the platform stores bcrypt-hashed credentials in MongoDB with 10 salt rounds. Upon successful login, a unique JWT is generated and issued to the user.

The JWT, containing essential user information such as `UserID` and `Role`, is securely signed using a secret key known only to the server. Whenever a user interacts with the platform, the JWT is included in the `Authorization` header of each request, and the backend validates it before granting access to protected routes such as book addition or loan management. JWTs have a finite expiration time, after which users must log in again.

5.4 TEST CASES

Test cases are essential for evaluating the functionality of the application, including user authentication, debounced searching, book borrowing, automated copy counting, and fine assessment.

5.4.1 TEST CASE I

Scenario: User logs in with incorrect credentials.

Expected Output: Login fails, and an error message is displayed stating "Invalid email or password".

Actual Output: Login attempt fails, and the user is prompted to re-enter correct credentials.

5.4.2 TEST CASE II

Scenario: Member borrows an available book (Eloquent JavaScript with 6 available copies).

Expected Output: Loan document is created with status "borrowed", and availableCopies decrements to 5.

Actual Output: Loan record successfully generated; available copies updated to 5 immediately.

5.4.3 TEST CASE III

Scenario: Member tries to add a new book without staff privileges.

Expected Output: Access denied with HTTP 403 Forbidden; system blocks unauthorized modification.

Actual Output: Server blocks the request with "User role 'Member' is not authorized to access this resource".

Table 5.1 Test Cases Summary

Test ID	Module	Scenario	Status
TC-01	Auth	Invalid password rejected	PASS
TC-02	Auth	Valid login returns JWT	PASS
TC-03	Search	Debounced search fires after 400ms	PASS
TC-04	Circulation	Borrow decrements available copies	PASS
TC-05	Circulation	Return increments available copies	PASS
TC-06	Security	Role guard blocks unauthorized action	PASS

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The Library Portal provides a comprehensive, modern platform for organizing academic library inventories and managing circulation workflows effectively. By offering features such as debounced search, category filtering, role-based dashboards for Members, Librarians, and Admins, and automated stock synchronization, the system enhances transparency, accountability, and operational ease.

Its intuitive interface and REST-based architecture allow the platform to maintain accurate records and provide a seamless experience for both patrons and staff. Overall, the project successfully integrates book management, lending tracking, member administration, and in-app notifications into a single, cohesive, eco-friendly digital platform.

6.2 FUTURE WORK

1. Barcode & RFID Scanner Integration

Integrating handheld USB barcode or RFID scanners at the front circulation desk will allow librarians to issue and return physical books in seconds with zero manual typing.

2. Digital eBook & PDF Reader

Embedding an in-browser PDF reader will permit patrons to read open-access research papers and digital publications directly within the web portal.

3. Automated SMS & Email Reminders

Future enhancements could include automated SMS or email reminders dispatched 48 hours before an item's due date.

4. Online Fine Payment Gateway

Incorporating digital payment gateways (e.g. Razorpay or UPI) would allow students to pay overdue book penalties directly from their profile dashboard.

CHAPTER 7

APPENDICES

APPENDIX I — SOURCE CODE

useDebounce.js (Custom Hook for Search Performance)

```
import { useState, useEffect } from "react";

export function useDebounce(value, delay = 400) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cancel timeout if value updates before delay expires
    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}

export default useDebounce;
```

loanController.js (Circulation & Stock Update Logic)

```
// Issue book to member & decrement stock count
const issueBook = async (req, res) => {
  const { bookId, memberId, durationDays = 14 } = req.body;
  const book = await Book.findById(bookId);
  if (!book || book.availableCopies <= 0) {
    return res.status(400).json({ message: "No copies available" });
  }

  const dueDate = new Date();
  dueDate.setDate(new Date().getDate() + Number(durationDays));

  const loan = await Loan.create({
    book: book._id,
    member: req.user.role === 'Member' ? req.user._id : memberId,
    dueDate,
    status: 'borrowed',
  });

  book.availableCopies -= 1;
  await book.save();
}
```

```
res.status(201).json(loan);  
};
```

APPENDIX II — SCREENSHOTS

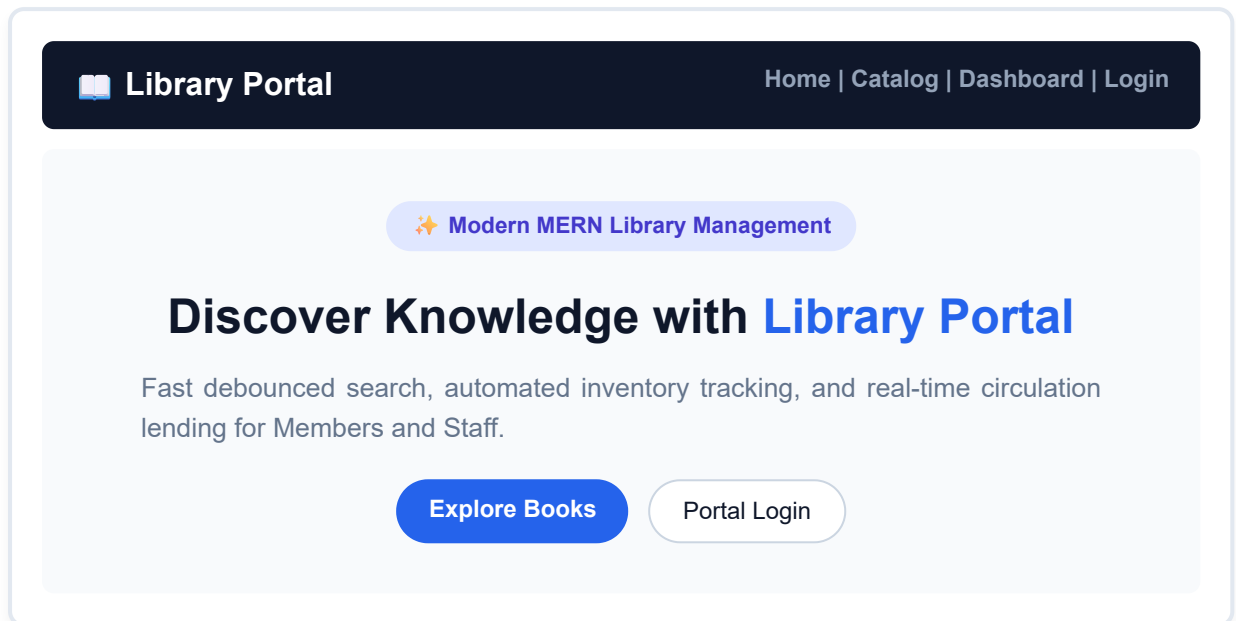


Fig. A.2.1. Home Page

Portal Login

Sign in to access your library dashboard

 **1-Click Demo Login:** Admin Librarian Member

Email Address

member@library.com

Password

.....

Sign In

Fig. A.2.2. Login Page

 **Library Catalog Directory**

 Debounced Search: Title, Author, ISBN...

All Computer Science Literature

Fig. A.2.3. Books Directory Page

Book Cover

Computer Science

4 Available

Clean Code: A Handbook of Agile Software Craftsmanship

By Robert C. Martin • ISBN: 978-0132350884 • 2008

Even bad code can function. But if code isn't clean, it can bring a development organization to its knees. This volume teaches software craftsmanship and clean refactoring patterns.

 Borrow This Book

Fig. A.2.4. Book Details & Checkout Page

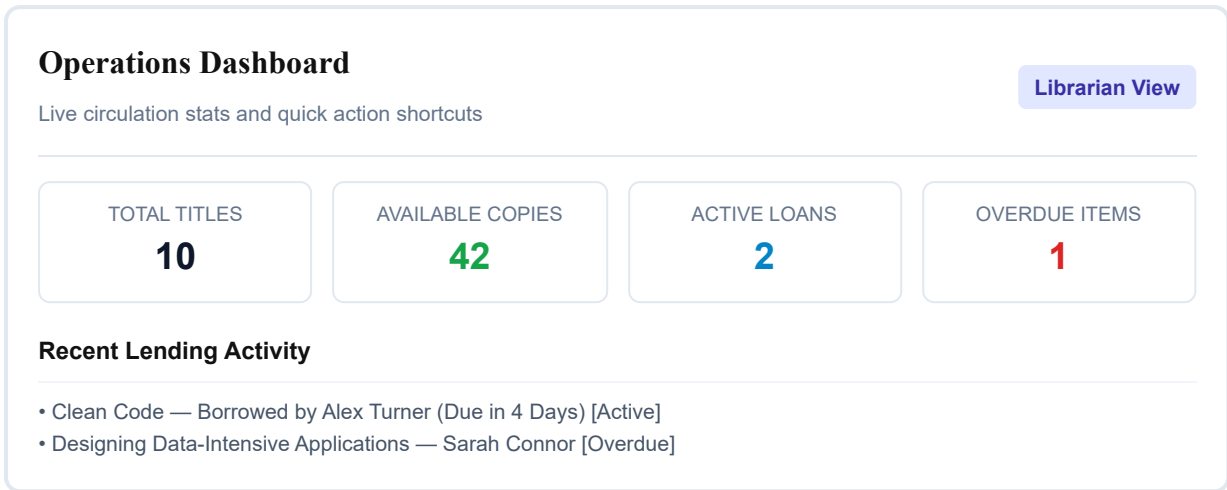


Fig. A.2.5. Dashboard Page

Circulation & Loan Ledger

[+ Issue Book](#)

Book	Borrower	Due Date	Status	Action
Clean Code	Alex Turner	20/09/2026	borrowed	Return
Designing Data-Intensive Apps	Sarah Connor	10/09/2026	overdue	Return

Fig. A.2.6. Loan Management Page

REFERENCES

Web References

- [1] React Official Documentation: <https://react.dev/learn>
- [2] React Router Documentation: <https://reactrouter.com/>
- [3] Node.js Official Documentation: <https://nodejs.org/en/docs>
- [4] Express.js Official Documentation: <https://expressjs.com/>
- [5] MongoDB Official Documentation: <https://www.mongodb.com/docs/>
- [6] Mongoose Official Documentation: <https://mongoosejs.com/docs/>
- [7] JSON Web Token (JWT) Official Documentation: <https://jwt.io/introduction/>
- [8] Axios Documentation: <https://axios-http.com/docs/intro>

Book References

- [1] Robert C. Martin (2008), *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall
- [2] Kyle Simpson (2015), *You Don't Know JS (Book Series)*, O'Reilly Media
- [3] Ethan Brown (2019), *Web Development with Node and Express*, O'Reilly Media
- [4] Stoyan Stefanov (2013), *React: Up and Running: Building Web Applications*, O'Reilly Media
- [5] Kristina Chodorow (2013), *MongoDB: The Definitive Guide*, O'Reilly Media